

Actividad #2

Luis Felipe Sierra

Instructor

Luis Fernando

CENTRO DE TECNOLOGIA DE LA

MANUFACTURA AVANZADA

2025

Preguntas de Introducción:

¿Qué es un diagrama?

Un diagrama es una representación visual que muestra información, ideas, procesos o relaciones de forma clara y ordenada. Se utilizan para facilitar la comprensión de conceptos complejos mediante formas, líneas, símbolos y textos. Los diagramas pueden mostrar cómo funciona un sistema, cómo se conectan diferentes elementos, o cómo se organiza una estructura. Por ejemplo, un diagrama de flujo muestra los pasos de un proceso, y un diagrama de base de datos representa las tablas y sus relaciones. En resumen, un diagrama ayuda a ver y entender mejor la información.

¿Qué tipos de diagramas conoce o ha utilizado?

Los diagramas que conozco y he utilizado son: mapas mentales, mapas conceptuales, diagrama circular y el diagrama de ven ¿Cómo se estructura un diagrama?

- Título:

Indica de qué trata el diagrama, para que el lector entienda el tema desde el inicio.

- Elementos **o componentes**:

Son las partes principales que se quieren representar, como procesos, ideas, tablas, clases o actores (dependiendo del diagrama).

- Conectores **o relaciones**:

Líneas, flechas o símbolos que muestran cómo se relacionan o interactúan los elementos entre sí.

- Símbolos **o formas**:

Cada tipo de diagrama usa formas específicas (rectángulos, círculos, rombos, etc.) con significados propios.

Por ejemplo, en un diagrama de flujo:

Rectángulo = proceso

Rombos = decisiones

Flechas = dirección del flujo

- Texto **o etiquetas**:

Palabras breves que explican cada componente o relación dentro del diagrama.

- Orden y **disposición lógica**:

Los elementos deben colocarse de manera que el diagrama se lea fácilmente (de arriba hacia abajo o de izquierda a derecha, según el caso)

¿Cómo se construye un diagrama?

- Define **el objetivo**:

¿Qué quieres mostrar con el diagrama? ¿Un proceso, una relación entre elementos, una estructura?

- Reúne **la información**:

Identifica los elementos clave (por ejemplo, pasos de un proceso, tablas de una base de datos, ideas principales, etc.).

- Elige **el tipo de diagrama adecuado**:

Usa un diagrama de flujo para procesos, uno entidad-relación para bases de datos, un mapa mental para ideas, etc.

- Organiza **los elementos**:

Piensa en el orden lógico o jerárquico. Por ejemplo, de arriba hacia abajo, de izquierda a derecha o del centro hacia afuera.

- Dibuja **los elementos y relaciones**:

Usa formas (rectángulos, círculos, rombos) para representar ideas, acciones o entidades.

Usa líneas o flechas para conectar los elementos y mostrar cómo se relacionan.

- Agrega **etiquetas o texto**:

Escribe nombres, descripciones breves o acciones dentro de las formas o cerca de las líneas, para que se entienda qué representa cada parte.

- Revisa y **ajusta**:

Asegúrate de que todo se entienda fácilmente. Verifica si falta información o si hay formas mejor de organizarlo.

- Usa **herramientas si es necesario**:

Puedes hacerlo a mano o usar programas como **Lucidchart**, **Draw.io**, **Microsoft Visio**, **PowerPoint**, **Word**, o incluso **papel y lápiz**

¿Cuál es el objetivo de usar diagramas?

El **objetivo de usar diagramas** es representar información de forma **visual, clara y organizada** para facilitar la comprensión, el análisis y la comunicación de ideas, procesos o estructuras. Los diagramas ayudan a:

Simplificar conceptos complejos que serían difíciles de entender solo con texto.

Visualizar relaciones entre elementos (por ejemplo, en una base de datos o en un sistema).

Explicar procesos paso a paso, como en los diagramas de flujo.

Planificar y diseñar proyectos o sistemas antes de desarrollarlos.

Comunicar ideas rápidamente a otras personas, como en presentaciones o reuniones.

Detectar errores o mejoras en procesos, estructuras o diseños.

¿Por qué consideras que deben usarse diagramas en el diseño de soluciones de software?

Los diagramas son esenciales en el diseño de soluciones de software porque permiten visualizar y entender cómo funcionará el sistema antes de desarrollarlo. Facilitan la comunicación entre el equipo, ayudan a detectar errores tempranamente, organizan mejor el trabajo y sirven como guía durante todo el proyecto.

Investigue:

¿Cuáles son los tipos de diagrama más relevantes en UML?

Diagrama de comunicación

Diagrama de despliegue

- Diagrama **de casos de uso**:

Muestra las **funciones principales** del sistema desde el punto de vista del usuario.

Representa actores (usuarios u otros sistemas) y los casos de uso (acciones que pueden realizar).

Es útil para entender qué debe hacer el sistema.

- Diagrama **de clases**:

Representa la **estructura del sistema**, mostrando las clases, sus atributos, métodos y relaciones entre ellas.

Es esencial para el diseño orientado a objetos y para organizar el código.

- Diagrama **de secuencia**:

Muestra cómo los **objetos del sistema interactúan entre sí en el tiempo**.

Es útil para ver el orden en que se realizan las acciones o mensajes entre componentes.

- Diagrama **de actividades**:

Representa **procesos o flujos de trabajo** paso a paso.

Se parece a un diagrama de flujo y es útil para modelar lógica de negocio o algoritmos.

¿Cómo representar instancias de las clases en UML?

En UML, las **instancias de una clase** se representan con **diagramas de objetos**, los cuales muestran **objetos específicos** (instancias) creados a partir de una clase. Estos diagramas

reflejan los **valores concretos** que tienen los atributos de la clase en un momento determinado.

+-----+

| Persona |

+-----+

| - nombre: String |

| - edad: int |

| - genero: String |

+-----+

| +saludar(): void |

| +cumplirAnios(): void |

+-----+

Nombre de la clase: Persona

Atributos: nombre, edad, genero

Métodos: saludar (), cumplirAnios()

: Persona nombre =

"Luis" edad = 25

genero = "Masculino"

Aquí se representa **un objeto específico** de la clase `Persona`.

El nombre del objeto puede escribirse como `luis:Persona` si se quiere nombrar la instancia.

Se listan los valores reales de sus atributos.

¿UML permite incluir notas, cómo, para qué?

Si, **UML permite incluir notas o comentarios** en los diagramas, y son muy útiles para **aclarar información, agregar explicaciones o dar contexto adicional** que no forma parte del modelo estructural o de comportamiento.

¿Cómo se representan las notas en UML?

Las **notas** se dibujan como un **rectángulo con la esquina superior derecha doblada**, como si fuera una hoja doblada.

Se pueden **conectar con una línea punteada** al elemento (clase, objeto, relación, etc.) al que se refieren.

¿Para qué se usan las notas en UML?

Agregar explicaciones o aclaraciones:

Por ejemplo, explicar por qué una clase tiene ciertos atributos o por qué se toma una decisión de diseño.

Documentar restricciones o reglas:

Como condiciones que no se pueden modelar gráficamente directamente, por ejemplo:

"La edad no puede ser negativa".

Anotar decisiones técnicas o pendientes:

Por ejemplo: “Este componente aún está en revisión” o “Se planea reemplazar esta clase en la siguiente versión”.

Facilitar la comunicación entre el equipo:

Las notas permiten que otros desarrolladores, diseñadores o clientes entiendan mejor el propósito de ciertos elementos del diagrama.

¿Qué es un método constructor y para qué se utiliza?

Un **método constructor** es una función especial que se utiliza en programación orientada a objetos para **crear e inicializar un objeto** a partir de una clase. Su función principal es **asignar valores iniciales** a los atributos del objeto cuando se crea una instancia.

¿Para qué se utiliza?

Para **asignar valores por defecto o personalizados** a los atributos del objeto.

Para **asegurar que el objeto se cree con datos válidos** desde el inicio.

Para ejecutar **acciones necesarias al momento de crear el objeto**, como conectar a una base de datos o cargar datos.

¿Qué tipos de Relaciones pueden especificarse entre clases?

En UML, entre las clases se pueden especificar varios **tipos de relaciones** que representan cómo interactúan o se conectan entre sí. Las más comunes son:

1. Asociación

Representa una **relación estructural** entre dos clases.

Ejemplo: una clase `Profesor` está asociada a una clase `Curso` porque un profesor puede dictar varios cursos.

2. Agregación (Asociación con sentido de todo-parte)

Indica que una clase **contiene** a otras, pero las partes **pueden existir por separado**.

Se representa con un **rombo blanco**.

Ejemplo: una clase `Departamento` puede contener varios `Empleados`, pero los empleados pueden existir aunque el departamento desaparezca.

3. Composición

Es una forma más fuerte de agregación: las partes **no pueden existir sin el todo**.

Se representa con un **rombo negro**.

Ejemplo: una clase `Casa` tiene `Habitaciones`, pero si la casa se destruye, las habitaciones también desaparecen.

4. Herencia o Generalización

Una clase **hereda atributos y métodos** de otra.

Se representa con una **flecha vacía apuntando a la clase padre**.

Ejemplo: `Empleado` hereda de `Persona`.

5. Dependencia

Una clase **usa** a otra temporalmente, pero no la contiene ni la hereda.

Se representa con una **línea punteada con flecha**.

Ejemplo: una clase `Factura` depende de `Cliente` para generar los datos, pero no lo almacena permanentemente.

ACTIVIDAD 1: DIAGRAMA DE CASOS DE USO

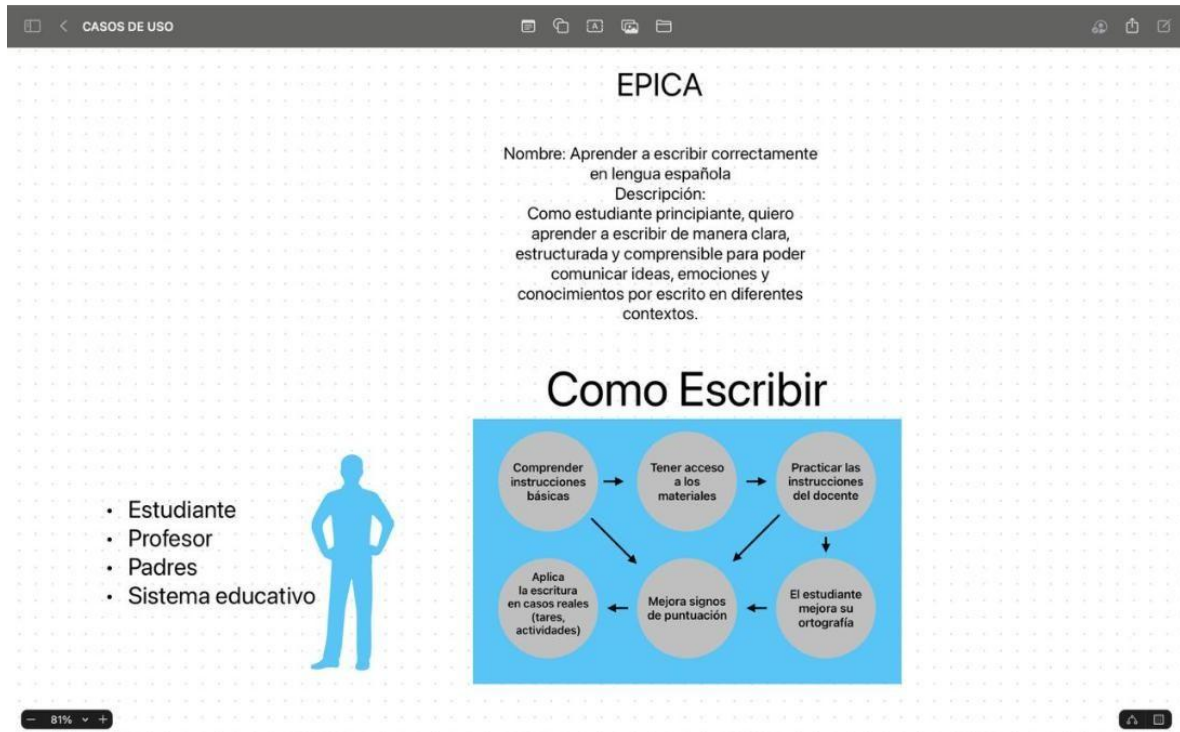
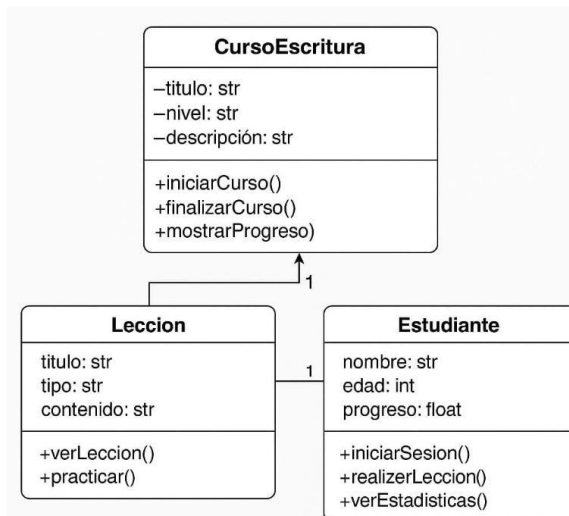


DIAGRAMA DE CLASES



GLOSARIO:

UML:

Es un lenguaje de modelado estándar que se utiliza para visualizar, especificar, construir y documentar los componentes de un sistema de software, especialmente orientado a objetos.

POO:

Paradigma de programación basado en el uso de "objetos", que son instancias de clases, para estructurar el software y sus interacciones.

Objeto:

Entidad que representa una instancia de una clase. Tiene atributos (estado) y métodos (comportamiento).

Clase:

Molde o plantilla que define las características (atributos) y comportamientos (métodos) comunes de un conjunto de objetos.

Modelo:

Representación simplificada y abstracta de la realidad que ayuda a entender, diseñar y construir un sistema.

Atributo:

Propiedad o característica de un objeto, definida en su clase. Por ejemplo, un objeto "Auto" puede tener atributos como color o velocidad.

Alcance:

Define el contexto o visibilidad de una variable, atributo o método dentro del código (público, privado, protegido, etc.).

Parámetro:

Variable que se pasa a una función o método para proporcionar información adicional que influya en su ejecución.

Agregación:

Tipo de relación entre clases donde una clase contiene a otra como parte, pero ambas pueden existir independientemente.

Herencia:

Mecanismo mediante el cual una clase (subclase) puede heredar atributos y métodos de otra clase (superclase), promoviendo la reutilización del código.

Polimorfismo:

Capacidad de un objeto de adoptar diferentes formas, lo que permite que métodos con el mismo nombre se comporten de manera distinta según el contexto o el tipo de objeto.

Extensión:

Relación en UML que permite añadir comportamientos opcionales a un caso de uso o ampliar funcionalidades de una clase.

Función:

Bloque de código que realiza una tarea específica y puede recibir parámetros y devolver un resultado.

Relación:

Conexión entre dos o más elementos (como clases o casos de uso) que indica cómo interactúan o se asocian entre sí.

Abstracción:

Principio de ocultar los detalles internos de implementación y mostrar solo lo esencial para reducir la complejidad.

Sobrecarga:

Capacidad de definir múltiples métodos o funciones con el mismo nombre pero con diferentes parámetros (tipo, número o ambos).

